Python Engineering - Reference Guide

Class 13 - Classes & Objects: OOP syntax reference

This guide is intentionally more explanatory than earlier notes. The goal is to build a clear mental model of what Python does when objects are created and used.

1. class

What it does: defines a new object type, including stored data and available operations.
What it produces: a class that can later be instantiated.

```
class VerticalState:
    ...
```

Think of the class as a blueprint, not yet a concrete physical state.

2. Object / instance

An object is a concrete realization of a class. VerticalState(...) creates an instance.

```
earth_state = VerticalState(100.0,20.0)
moon_state  = VerticalState(100.0,20.0,g=1.62)
```

Both are VerticalState objects but store independent data.

3. __init__()

What it does: initializes an object immediately after creation.
What it receives: self plus initialization arguments.
What it returns: normally no explicit return.

```
def __init__(self,h,v,t=0.0,g=9.81):
    self.h=h
    self.v=v
    self.t=t
    self.g=g
```

__init__ prepares each object's initial state.

4. self

Meaning: self refers to the particular object currently using the method.

```
earth_state.specific_energy()
# self -> earth_state

moon_state.specific_energy()
# self -> moon_state
```

The same method code can work on different objects because self identifies the current one.

5. Parameter h vs attribute self.h

h is a temporary incoming parameter. self.h is persistent data stored on the object.

```
self.h = h
# h -> parameter
# self.h -> stored attribute
```

The parameter is local to the call; the attribute belongs to the object.

6. Attribute

An attribute is data associated with an object: self.h, self.v, self.t, self.g.

```
earth_state.h
earth_state.v
```

```
earth_state.g
```

Attributes describe object state.

7. Method

A method is a function defined inside a class and intended to operate on the object, usually receiving self first.

```
def specific_energy(self):
    return self.g*self.h + 0.5*self.v**2
```

Explicit input: none. Implicit input: self. Output: specific energy.

8. Method that changes state

euler_step(dt) changes h, v, and t stored on the object.

```
earth_state.euler_step(0.1)
# object now stores new h, v and t
```

After the call, the same object contains different values.

9. isinstance()

What it does: checks whether an object belongs to a class/type.
Returns: True or False.

```
isinstance(earth_state, VerticalState)
# True
```

Useful for type inspection and validation.

10. vars() as a learning tool

For simple objects, vars(object) exposes the instance attribute dictionary.

```
vars(earth_state)
# {'h':100.0,'v':20.0,'t':0.0,'g':9.81}
```

Not required for EX21; useful to visualize stored state.

11. Object call vs class call

When a method is called through an object, Python supplies self automatically.

```
earth_state.specific_energy()
# usual form

VerticalState.specific_energy(earth_state)
# self explicit
```

This explains why self appears in the definition but not in the normal object call.

12. Mental summary

class = defines the type/blueprint.
object = concrete instance.
__init__ = sets initial attributes.
self = reference to the current object.
attribute = data stored on the object.
method = function associated with the object.
mutation = change of stored object state.